Teoría de Algoritmos
(75.29) Curso Buchwald - Genender

Trabajo Práctico 1
Algoritmos Greedy

8 de abril de 2026

Paula Romano	Matias Urbani	Juan Contino
112609	113739	113861

Índice

1. Introducción **3**

2. Análisis del Problema: **3**

3. Demostración de Optimalidad: **4**

4. Algoritmo: **5**

4.1. Código: 5

4.2. Complejidad: 5

4.3. Por que es un algoritmo Greedy: 5

5. Análisis de tiempo: **6**

6. Análisis de optimalidad: **6**

7. Pruebas: **7**

7.1. Caso base: 7

7.2. caso variados: 7

7.3. caso Ayudantes iguales: 7

7.4. caso Scaloni iguales: 8

7.5. caso Mismas proporciones: 8

8. Mediciones: **9**

9. Conclusiones: **11**

1. Introducción

Tenemos que ayudar a Scaloni a analizar los próximos n rivales de la selección campeona del mundo. Como técnico perfeccionista, quiere analizar videos de cada uno de los rivales. Recibió un compilado por cada rival, y necesita hacer un análisis muy detallado, lo cual le implica tomar apuntes, analizar tácticas, ver cuándo hay que hacerle un masaje a Messi, etc... Para que el análisis sea detallado, cada compilado no lo revisa únicamente él, sino también un ayudante.

El análisis del rival i le toma Si de tiempo a Scaloni, y luego Ai al ayudante (independientemente de cuál ayudante lo vea). Lo bueno, es que después de los grandes logros obtenidos, Scaloni cuenta con n ayudantes (es decir, la misma cantidad que rivales), que pueden ver los videos completamente en paralelo. Siempre los ayudantes podrán ver los videos después que Scaloni haya terminado de verlo y analizarlo como corresponde (esto no lo delega). Cuando llega la hora que un ayudante lo vea, puede ser cualquiera, pero sólo uno lo verá (no hay ganancia en que dos lo vean).

El DT necesita que los rivales estén todos con sus correspondientes análisis lo antes posible, y por eso te pide que lo ayudes. Dice que confía en vos. Sabe que no lo vas a dejar tirado.

2. Análisis del Problema:

El problema que se plantea en el Trabajo Practico es el siguiente:

Recibiendo un numero n que representa la cantidad de rivales a los que se debe enfrentar la selección, un arreglo S, donde la posición S[i] representa el tiempo que le toma a Scaloni analizar al i-esimo rival, y un arreglo A, donde la posición A[i] representa el tiempo que le toma a un ayudante analizar al i-esimo rival.

Se debe analizar a todos los n rivales, donde primero los analizara Scaloni y luego algún ayudante, pudiendo varios ayudantes analizar a varios rivales en paralelo, mientras que Scaloni solo puede analizar a los rivales de uno en uno, y los ayudantes solo analizaran rivales que Scaloni ya haya analizado

Se pide obtener un orden para analizar a los n rivales, de modo que el tiempo total sea el mínimo posible

Proponemos como algoritmo para solucionar este problema ordenar los rivales por el tiempo que le toma a los ayudantes analizarlo, de manera descendente, y devolverlos en ese orden

3. Demostración de Optimalidad:

Sean los rivales $R_1, R_2...R_n$, donde el rival R_i se corresponde con los tiempos de las posiciones S_i y A_i

Sea O un orden de los n rivales

Sean T_i el tiempo parcial hasta el i-esimo rival , tal que

$$T_i = \sum_{k=1}^{i} S_k + A_i \tag{1}$$

El tiempo total de analizar a los n rivales en el orden O es

$$T_o = max(T_1, T_2, ..., T_n) \tag{2}$$

Se puede demostrar que nuestro algoritmo encuentra la solución optima usando la técnica de las inversiones

Supongamos que nuestro algoritmo devuelve un orden O_a, de la forma

$$O_a = (R_1, R_2, ..., R_i, R_j, ...R_n) \tag{3}$$

donde para cada par de Rivales R_k y R_t con $k < t$

$$A_k > A_t \tag{4}$$

y el orden optimo O_p fuera de la forma

$$O_p = (R_1, R_2, ..., R_j, R_i, ...R_n) \tag{5}$$

es decir, tuviera una inversión de los elementos R_i y R_j.

Los tiempos parciales de los rivales R_i y R_j en O_a son

$$T_i = \sum_{k=1}^{i} S_k + A_i \tag{6}$$

$$T_j = \sum_{k=1}^{i} S_k + S_j + A_j \tag{7}$$

Los tiempos parciales de los rivales R_j y R_i en el O_p son

$$T'_j = \sum_{k=1}^{j} S_k + A_j \tag{8}$$

$$T'_i = \sum_{k=1}^{j} S_k + S_i + A_i \tag{9}$$

Para que el orden optimo pudiera ser mejor que el orden planteado entonces $T_{Op} < T_{Oa}$

Como los tiempos parciales de los elementos que no están entre R_i y R_j son iguales en ambos ordenes, no puede haberse mejorado el tiempo con ninguno de ellos

Sean los máximos

$$M_a = max(T_i, T_j) \tag{10}$$

$$M_p = max(T'_j, T'_i) = max(\sum_{k=1}^{j} S_k + A_j, \sum_{k=1}^{j} S_k + S_i + A_i) = T'_i \tag{11}$$

Para que $T_{Op} < T_{Oa}$ debe darse que

$$M_p < M_a => T'_i < M_a \tag{12}$$

es decir, debe darse simultáneamente que

$$T'_i < T_i \tag{13}$$

$$T'_i < T_j \tag{14}$$

y

$$T'_i < T_i => \sum_{k=1}^{i} S_k + S_j + A_i < \sum_{k=1}^{i} S_k + A_i => S_j < 0 \tag{15}$$

y por condición del problema, S_j no puede ser menor que 0 para ningún j, por lo que es un absurdo.

Como O_p no puede ser mejor que O_a, queda demostrado que nuestro algoritmo obtiene siempre la solución optima al problema

4. Algoritmo:

4.1. Código:

A continuación se muestra el código de solución Greedy del problema.

```
def tiempo_minimo(rivales):
    #rivales = (s_i a_i)
    rivales_ordenados = sorted(rivales, key=lambda x: x[1], reverse=True)
    tiempo_scaloni = 0
    tiempo_total = 0
    for s_i, a_i in rivales_ordenados:
        tiempo_scaloni += s_i
        final = tiempo_scaloni + a_i
        tiempo_total = max(tiempo_total, final)
    return rivales_ordenados, tiempo_total
```

4.2. Complejidad:

Procesar los datos de entrada: O(n)
Ordenarlos: O(n log(n))
Total: O(n log(n))

4.3. Por que es un algoritmo Greedy:

Regla Greedy: En cada iteración se agrega el rival con el mayor tiempo del ayudante
Optimo Local: En cada iteración, el algoritmo elige el elemento no utilizado que tiene el mayor tiempo de ayudante, esperando que el tiempo de ese ayudante se compense con los tiempos de Scaloni y de los ayudantes de los rivales siguientes, y que colocando los tiempos mas grandes primero, estos no aporten al tiempo global

5. Análisis de tiempo:

EL tiempo de ejecución del algoritmo depende unicamente de la cantidad de rivales y no de los valores específicos de S_i y A_i. Esto se debe a que las operaciones consisten en un ordenamiento y un recorrido lineal, y su complejidad es independiente a los valores dados.

Se realizaron ejecuciones del programa con distintos tamaños de entrada, obteniendo los siguientes resultados:

```
$ python3 tp1.py 3elem.txt
Resultado: 10
Tiempo de ejecución: 0.000006 segundos

$ python3 tp1.py 10elem.txt
Resultado: 29
Tiempo de ejecución: 0.000010 segundos

$ python3 tp1.py 100elem.txt
Resultado: 5223
Tiempo de ejecución: 0.000037 segundos

$ python3 tp1.py 10000elem.txt
Resultado: 497886735
Tiempo de ejecución: 0.002555 segundos
```

Se observa que el tiempo de ejecución crece con el tamaño de entrada, de forma consistente con la complejidad $O(n \log n)$.

6. Análisis de optimalidad:

Los valores de A_i son los que determinan el orden del algoritmo, influyendo directamente en el tiempo de finalización de cada análisis. Cuanto mayor es A_i mayor impacto tiene en el tiempo total, por lo que conviene procesarlos primero. La variabilidad en los valores no afecta la optimalidad, solamente define un orden.

Los valores de S_i no influyen en el ordenamiento del algoritmo. Afectan al tiempo total, se van acumulado, sin cambiar que orden es mejor.

En conclusión, el orden optimo depende solamente de A_i, ya que son los que determinan el total del tiempo. Los valores de S_i no modifican el orden, por lo que el algoritmo es siempre optimo para cualquier variación de valores

Para validar esto de manera empírica, se realizaron pruebas comparando el resultado del algoritmo Greedy con otros órdenes posibles, como ordenar los rivales según S_i y órdenes aleatorios. Para distintos tamaños de entrada, se obtuvieron los siguientes resultados:

```
$ python3 tp1.py 3elem.txt
Greedy: 10
Por S_i: 10
Aleatorio: 17

$ python3 tp1.py 10elem.txt
Greedy: 29
Por S_i: 33
Aleatorio: 29

$ python3 tp1.py 100elem.txt
```

```
Greedy: 5223
Por S_i: 5240
Aleatorio: 5226

$ python3 tp1.py 10000elem.txt
Greedy: 497886735
Por S_i: 497936451
Aleatorio: 497956158
```

Se observa que el algoritmo Greedy obtiene siempre un resultado menor o igual al de los demás órdenes. En algunos casos puede coincidir con otros órdenes, pero nunca es superado.

Estos resultados experimentales son consistentes con el análisis teórico y respaldan que el algoritmo propuesto produce una solución óptima.

7. Pruebas:

Se analizaron varios casos para corroborar que el algoritmo es optimo en determinadas circunstancias:

7.1. Caso base:

Scaloni	Ayudante
10	7
12	4
9	5

El tiempo total es 35
En un caso normal sin valores demasiado grandes o pequeños el tiempo de los n rivales, es igual al tiempo parcial del ultimo rival, es decir $\sum_{k=1}^{n} S_k + A_n$, y en este caso eso se cumple, pues el tiempo total es $35 = 10 + 12 + 9 + 4$

7.2. caso variados:

Scaloni	Ayudante
10	99
70	20
80	5
10	1

El tiempo total es 171
En un caso muy variado el tiempo de los ayudantes mas grandes se compenso con el tiempo de los Scaloni mas grandes, resultando en un tiempo aproximadamente igual al caso base. En este caso da igual al caso base pues es $171 = 80 + 70 + 10 + 10 + 1$

7.3. caso Ayudantes iguales:

Scaloni	Ayudante
19	5
13	5
15	5
12	5
10	5

El tiempo total es 74
Con tiempos de ayudante iguales, el tiempo depende es igual a la suma de los valores de Scaloni, mas el tiempo de algun ayudante, en este caso $74 = 19 + 13 + 15 + 12 + 10 + 5$

7.4. caso Scaloni iguales:

Scaloni	Ayudante
10	2
10	4
10	6
10	11
10	12
10	9
10	3
10	4

El tiempo total es 82
Con tiempos de Scaloni iguales, el tiempo depende de que tan grande sean los valores de los ayudantes en relacion a el tiempo de Scaloni, si fueran mucho mas grandes, podria aumentar el tiempo de analizis de los n rivales

7.5. caso Mismas proporciones:

Scaloni	Ayudante
10	5
4	2
24	12
12	8
20	10

El tiempo total es 72

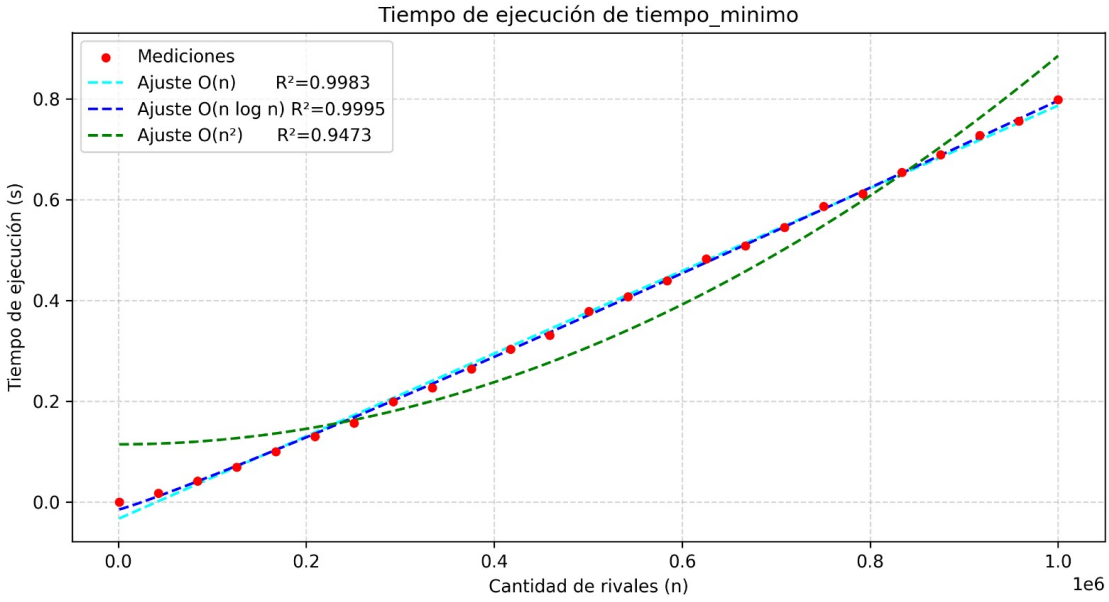
8. Mediciones:

Para corroborar la complejidad temporal O(n*log(n)), se generaron conjuntos de datos de distintos tamaños y distribuciones, los cuales se utilizaron para medir el tiempo de ejecución del algoritmo propuesto.
Los conjuntos generados para el ajuste de complejidad fueron creados con valores de S_i y A_i aleatorios en el rango [1, 1000], con tamaños de entre 1000 y 1000000 rivales.
Se ajustaron las siguientes curvas mediante el método de cuadrados mínimos:

$$F_1(x) = C_1 * x + C_2 \tag{16}$$

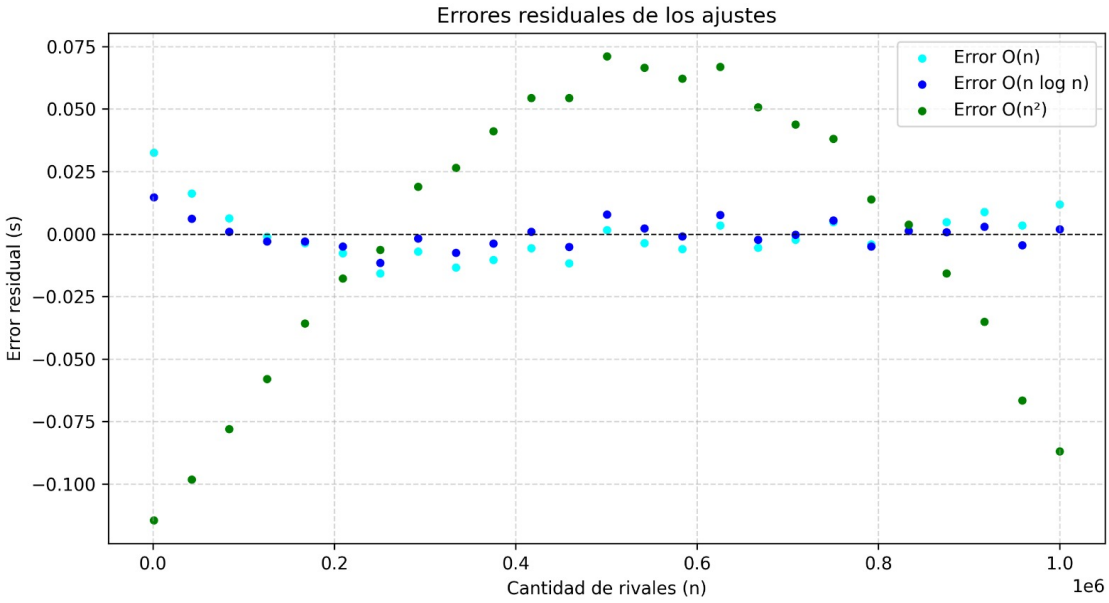
$$F_2(x) = C_1 * x * log(x) + C_2 \tag{17}$$

$$F_3(x) = C_1 * x^2 + C_2 \tag{18}$$



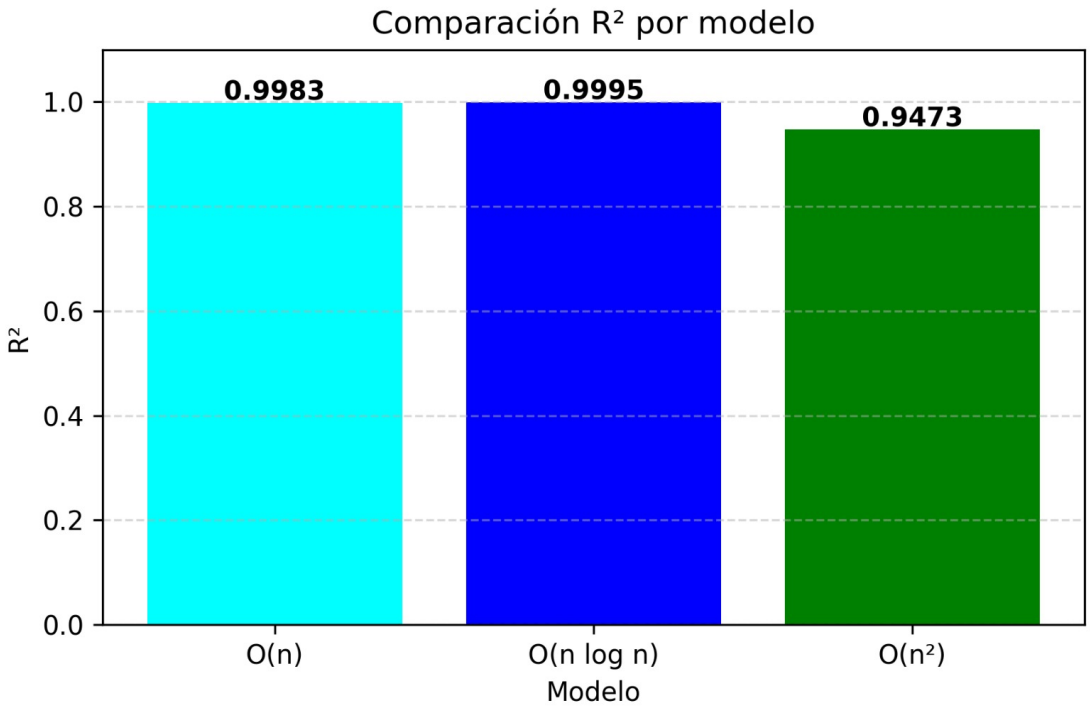
Se observa que la función O(n*log(n)) es la que mejor se ajusta a los datos medidos, con un $R^2 = 0{,}9995$. Si bien O(n) también presenta un R^2 alto (0.9983), visualmente se puede apreciar que O(n log n) sigue con mayor precisión. El modelo $O(n^2)$ queda claramente descartado, tanto por su R^2 considerablemente menor (0.9473) como por la divergencia visible en el gráfico a medida que n crece.

Para visualizar esto con mayor claridad, se graficaron también los errores residuales de cada ajuste:



Se observa que los errores del ajuste O(n log n) son los más pequeños y están distribuidos de forma uniforme alrededor de cero a lo largo de todo el rango evaluado. El ajuste O(n) presenta errores algo mayores pero igualmente acotados, mientras que $O(n^2)$ muestra errores crecientes, confirmando que no es el modelo correcto.

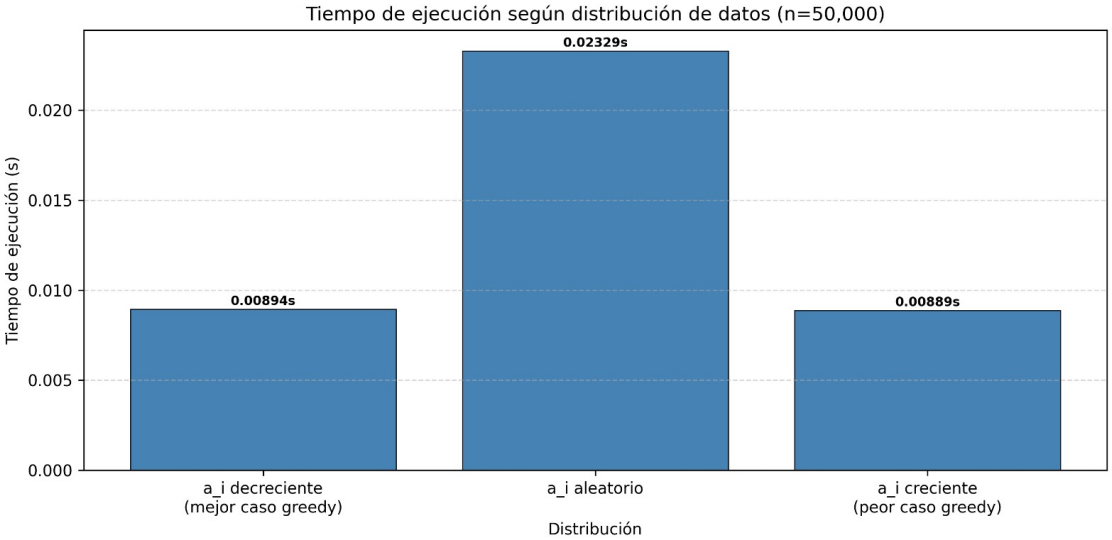
Esto se corrobora también mediante el coeficiente R^2, que indica qué proporción de la variación en los tiempos es explicada por cada modelo ($R^2 = 1$ indica ajuste perfecto, $R^2 = 0$ indica que el modelo no explica nada):



El modelo O(n log n) obtiene el mayor R^2 (0.9995).

Para analizar si la distribución de los datos de entrada afecta el tiempo de ejecución, se construyeron tres conjuntos de prueba con $n = 50{,}000$ rivales:

- A_i decreciente: los rivales ya vienen en el orden requerido.
- A_i aleatorio: distribución uniforme.
- A_i creciente: los rivales vienen en el orden inverso al requerido.



Los casos con A_i decreciente y A_i creciente tardan aproximadamente lo mismo (0.00894s y 0.00889s respectivamente), mientras que el caso aleatorio tarda considerablemente más (0.02329s). Esto se debe a que el ordenamiento interno utiliza sorted() de Python, que implementa Timesort. Este algoritmo escanea la lista en busca de sublistas ya ordenadas y las aprovecha: tanto la lista decreciente como la creciente son detectadas como listas completamente ordenadas (una en sentido directo y otra en sentido inverso), por lo que Timesort las procesa en O(n) en lugar del O(n log n) general. El caso aleatorio, en cambio, no presenta ningún orden previo aprovechable, por lo que Timesort debe realizar el trabajo completo.

Esta variación en los tiempos no afecta la optimalidad del algoritmo Greedy: independientemente de cómo vengan los datos de entrada, el resultado obtenido es siempre óptimo.

9. Conclusiones:

En este Trabajo Practico se analizo un problema que parece difícil a primera vista, pero al analizarlo cuidadosamente resulta en un algoritmo sencillo. Es fácil notar que colocar los Tiempos de los ayudantes mas grandes al final influiría negativamente en el Tiempo total, por lo que la opción de ordenarlos de mayor a menor surge naturalmente. Probando diferentes casos, se puede notar que los tiempos que demora Scaloni no tienen importancia si se ordena de esta manera, y la mayor dificultad radica en demostrar que esta sea la solución optima Un problema que inicialmente parece que no vaya a tener una solución optima y sencilla, pero resulta que al encararlo con un algoritmo Greedy, la solución optima surge naturalmente y el código no tiene mayores dificultades